

Batch adaptatif par inner-product test sur SVGD-EDA — bilan

16 septembre 2026

1 Objectif

Tester si le critère inner-product test (Bollapragada, Byrd & Nocedal, 2018) peut piloter dynamiquement la taille de batch Monte-Carlo (λ_a) par agent SVGD-EDA, plutôt qu'un λ_a fixe.

Critère, recalculé à chaque tour de doubling sur les λ_a échantillons déjà tirés :

- $z^{(i)} = a^{(i)} \cdot \nabla_{\theta} \log \pi_{\theta}(x^{(i)})$ — contribution par échantillon ($a^{(i)}$: avantage rang/fitness ; score en forme close $x - p$ ou onehot($x - p$)).
- $\hat{g} = \frac{1}{\lambda_a} \sum_i z^{(i)}$ — gradient estimé.
- $\widehat{\text{tr}\Sigma} = \frac{1}{\lambda_a - 1} \sum_i \|z^{(i)} - \hat{g}\|^2$ — dispersion (bruit Monte-Carlo).
- $\|\hat{g}\|_{\text{corr}}^2 = \max(\|\hat{g}\|^2 - \widehat{\text{tr}\Sigma}/\lambda_a, 0)$ — norme débiaisée du bruit.
- $\hat{g}^{\top} \Sigma \hat{g} = \|\hat{g}\|^4 \cdot \text{Var}(\hat{s}^{(i)})$, avec $\hat{s}^{(i)} = \langle z^{(i)}, \hat{g} \rangle / \|\hat{g}\|^2$ — variance projetée.
- $\lambda_{a,\text{req}} = \hat{g}^{\top} \Sigma \hat{g} / (\text{ip_tol}^2 \cdot \|\hat{g}\|_{\text{corr}}^4)$ — taille requise ; $+\infty$ si $\|\hat{g}\|_{\text{corr}}^2 = 0$.

Doubling ($\lambda_{a,\text{init}} \rightarrow 2 \times \rightarrow \dots \rightarrow \lambda_{a,\text{max}}$) jusqu'à $\lambda_a \geq \lambda_{a,\text{req}}$ ou $\lambda_{a,\text{max}}$. Si plafonné : shrinkage du pas SVGD, $\eta \leftarrow \eta \cdot \|\hat{g}\|_{\text{corr}}^2 / (\|\hat{g}\|_{\text{corr}}^2 + \widehat{\text{tr}\Sigma}/\lambda_a)$.

2 Implémentation

- Décision indépendante par (instance, agent) — pas d'agrégation sur le batch.
- $\lambda_{a,\text{init}} = 10$ par défaut (cf. §3.4 pour la sensibilité), $\lambda_{a,\text{max}}$ modeste (24-48).
- Contrat externe inchangé : `agent_lambdas` reste constant à $\lambda_{a,\text{max}}$, instances arrêtées tôt complétées par duplication de leurs échantillons réels.

3 Résultats — NK (rugosité contrôlée par K)

3.1 Score final vs baseline $\lambda_a = 10$ fixe

Budget=50000, 10 instances $\times$ 10 restarts.

	K=2	K=4	K=8
baseline	0.7477	0.7608	0.7542
meilleur adaptatif	0.7466 (ip5/lmax24)	0.7575 (ip5/lmax48)	0.7408 (ip5/lmax48)
écart	-0.15%	-0.4%	-1.8%

L'écart au baseline grandit avec K, de façon monotone.

3.2 Fraction plafonnée et shrinkage sur un run complet

Run complet (50000 évaluations, $\lambda_{a,\text{init}} = 10$, $\lambda_{a,\text{max}} = 48$), mesuré sur toutes les itérations cumulées. Colonnes : %capped / shrink médian.

dim	K	ip_tol=0.4	ip_tol=5	ip_tol=20	ip_tol=100
64	1	89.0% / 0.262	10.1% / 0.0045	9.7% / 0.0044	6.7% / 0.0028
64	2	92.5% / 0.221	12.5% / 0.0039	9.7% / 0.0042	7.8% / 0.0027
64	4	98.9% / 0.099	7.7% / 0.0000	8.6% / 0.0032	4.1% / 0.0000
64	8	99.8% / 0.0013	14.2% / 0.0000	8.9% / 0.0000	7.8% / 0.0000
256	8	100.0% / 0.0000	31.9% / 0.0000	17.9% / 0.0000	14.5% / 0.0000

- Rendements décroissants sur ip_tol : chute nette entre 0.4 et 5, puis effet marginal (5 → 20 → 100), car ip_tol n'intervient que si $\|\hat{g}\|_{\text{corr}}^2 > 0$.
- Résidu incompressible qui empire avec la dimension : même à ip_tol=100, 6-8% restent plafonnées à dim=64 et **14.5%** à dim=256 (K=8), avec shrink médian quasi nul.

3.3 Effet de la dimension : NK N=256, K=8

Même protocole, problème 4× plus grand (256 au lieu de 64) :

config	score
baseline rang, $\lambda_a = 10$	0.7235
adaptatif ip5, lmax48	0.6162
adaptatif ip20, lmax48	0.6401
adaptatif ip50, lmax48	0.6423
écart (meilleur cas)	−11.5%

Dégradation bien plus forte qu'à N=64 K=8 (−1.8%) alors que le %capped n'augmente que modérément — la rugosité (K) et la dimension (N) semblent se cumuler plutôt que jouer indépendamment.

3.4 Sensibilité à $\lambda_{a,\text{init}}$ (QUBO, rang, $\lambda_{a,\text{max}} = 48$)

$\lambda_{a,\text{init}}$	ip_tol=5	ip_tol=20
10	−301.60	−295.00
3	−299.20	−291.60

$\lambda_{a,\text{init}} = 3$ fait systématiquement moins bien que 10 : redescendre sous le λ_a connu-bon coûte plus qu'il ne rapporte en économie de budget.

4 Résultats — QUBO (dim=64, instance t=5)

Même reward des deux côtés (rang) — nb_instances_test=5, nb_restarts=2, budget=50000 ; **plus bas = meilleur** :

config	score
baseline rang, $\lambda_a = 10$	−304.80
adaptatif ip5, lmax24	−304.20
adaptatif ip5, lmax48	−301.60
adaptatif ip20, lmax24	−298.80
adaptatif ip20, lmax48	−295.00
adaptatif ip50, lmax24	−304.00
adaptatif ip50, lmax48	−300.60

Meilleur cas quasi identique au baseline (bruit d'un seul run) ; aucun cas ne le bat.

Note : un premier essai avait suggéré un gain massif, dû à un facteur confondu (reward différent entre adaptatif et baseline) ; une fois égalisé, le gain disparaît.

5 Diagnostic

1. Paysage combinatoire intrinsèquement rugueux (surtout K élevé) : $\|\hat{g}\|_{\text{corr}}^2$ reste souvent sous le bruit même avec beaucoup d'échantillons — régime courant, pas un cas rare de fin de convergence comme suppose le papier.
2. Doubling multiplicatif : phénomène tout-ou-rien, λ_a explose vite au plafond.
3. Shrinkage quasi total sur les instances plafonnées, gelant une fraction croissante de la population à mesure que K augmente.
4. SVGD-EDA tire sa force du nombre d'itérations (répulsion, moyennisation du bruit dans le temps), pas de la fiabilité de chaque pas — philosophie opposée à l'inner-product test.

6 Conclusion

Une fois isolé de tout facteur confondu, le batch adaptatif **n'égale jamais un λ_a fixe bien réglé** : quasi neutre sur les paysages peu rugueux et petits (NK $K=1-2$, QUBO), nettement dégradé sur les paysages rugueux et/ou grands (NK $K=8$, jusqu'à -11.5% à $N=256$). La dégradation s'aggrave avec la rugosité (K) et la dimension (N), et ces deux effets se cumulent. Le mécanisme fonctionne comme prévu par sa propre logique (plus de budget là où le signal est faible), mais cette logique ne s'aligne pas avec les besoins d'un optimiseur multi-agents itératif.